

Tut06

Anne

Anmerkung: geht bitte in die Vorlesung -> wir behandeln nur Teile

10 min Aufgabe lösen:

Aufgabe 8 – Datenübertragung

Stellen Sie sich das folgende Szenario vor. Auf Ihrem Computer befindet sich ein Verzeichnis, das 10 TB³ Daten enthält. Sie wollen dieses nun aus Berlin an einen Computer senden, der in Hamburg steht. Ihre Internetverbindung ermöglicht das Senden von Daten mit einer Rate von 1000 Mb/s, der Computer in Hamburg kann Daten mit 500 Mb/s empfangen.⁴ Mithilfe des Programms „ping“ haben Sie herausgefunden, dass die Paketumlaufzeit (engl. *round-trip time*) 1000 ms beträgt, wobei Sie davon ausgehen können, dass die Geschwindigkeit in beide Richtungen gleich ist.

- a) Finden Sie heraus, wie groß Latenz und Durchsatz der Datenübertragung sind. Wie lange wird es dauern, bis alle Daten vollständig übertragen wurden?
- b) Eine Kollegin schlägt Ihnen vor, die Daten auf eine Festplatte zu kopieren und direkt nach Hamburg zu fahren. Ihre Festplatte hat Schreib- und Lesegeschwindigkeiten von 1000 Mb/s, die Fahrt nach Hamburg dauert etwa drei Stunden. Was halten Sie von dem Vorschlag?

Info: mb/s -> Megabit

1 Byte = 8 Bit

1 TB = 1 000 000 MegaByte

- Latenz:
- Paketumlaufzeit = 1000 ms
- In eine Richtung = - 500 ms

Übertragung:

10 TB = 10 000 000 Megabyte

10 TB = 80 000 000 Mb

$80\,000\,000\text{Mb} / 500\text{ mb/s} + 500\text{ ms} = 160\,000,5\text{ s}$

2 667 min = 44,4 h

- Durchsatz:
- Bottleneck: Hamburg kann nur 500 Mb/s empfangen
- 500 Mb/s

Auto:

$80\,000\,000\text{ Mb} / 2\,000\text{ Mb/s} = 40\,000\text{ s}$

= 11,11h

22,2 h + 3h = 25,3 h

Aufgabe 8 – Datenübertragung

Stellen Sie sich das folgende Szenario vor. Auf Ihrem Computer befindet sich ein Verzeichnis, das 10 TB³ Daten enthält. Sie wollen dieses nun aus Berlin an einen Computer senden, der in Hamburg steht. Ihre Internetverbindung ermöglicht das Senden von Daten mit einer Rate von 1000 Mb/s, der Computer in Hamburg kann Daten mit 500 Mb/s empfangen.⁴ Mithilfe des Programms „ping“ haben Sie herausgefunden, dass die Paketumlaufzeit (engl. *round-trip time*) 1000 ms beträgt, wobei Sie davon ausgehen können, dass die Geschwindigkeit in beide Richtungen gleich ist.

a) Finden Sie heraus, wie groß Latenz und Durchsatz der Datenübertragung sind. Wie lange wird es dauern, bis alle Daten vollständig übertragen wurden?

b) Eine Kollegin schlägt Ihnen vor, die Daten auf eine Festplatte zu kopieren und direkt nach Hamburg zu fahren. Ihre Festplatte hat Schreib- und Lesegeschwindigkeiten von 2000 Mb/s, die Fahrt nach Hamburg dauert etwa drei Stunden. Die Festplatte des Computers in Hamburg ist genauso schnell. Was halten Sie von dem Vorschlag?

Verfügbarkeit (Availability)

- Wie zuverlässig ist System?
- Wir geben in Prozent an wie hoch die Verfügbarkeit ist
- $\text{Uptime} = ((\text{Gesamtzeit} - \text{Ausfallzeit}) / \text{Gesamtzeit}) * 100$

Availability %	Name	Downtime pro Jahr
99%	two nines	3,65 Tage
99,9%	three nines	8,77 Stunden
99,99%	four nines	52,6 Minuten

Aufgabe:

- Ein Online-Shop hatte im letzten Jahr folgende Ausfälle:
- **Januar:** 3 Stunden Ausfall
- **März:** 5 Stunden Ausfall
- **Juni:** 2 Stunden Ausfall
- **September:** 4 Stunden Ausfall
- **Dezember:** 1 Stunde Ausfall

- a) Berechne die **Uptime in Prozent** für das Jahr.
- b) Wie viele "Nines" hat das System? (99%, 99,9%, 99,99%?)

$$\text{Uptime} = ((\text{Gesamtzeit} - \text{Ausfallzeit}) / \text{Gesamtzeit}) * 100$$

- Uptime = $((8760 - 15) / 8760) * 100$
- = 99,829
- Zwischen two Nines und three Nines
- Wie viele Stunden dürfte das system Maximal down sein, um 3 Nines zu haben ?
- Maximal 8,76 h

XML

- Daten werden mit Tags beschrieben
- Jedes öffnende Tag braucht ein schließendes Tag
- -> wollen Daten strukturiert speichern (universelles Textformat)

```
<?xml version="1.0" encoding="UTF-8"?>

<BillingInformation>
  <Name>Right Plastic Products</Name>
  <BillingDate>2023-06-18</BillingDate>
  <Address>
    <Street>158 Edward st.</Street>
    <City>Brisbane</City>
    <State>QLD</State>
    <PostalCode>4000</PostalCode>
  </Address>
</BillingInformation>
```

2 Datenformate

Aufgabe 3 – XML Grundlagen

Gegeben sei das folgende, syntaktisch inkorrekte XML-Dokument. Korrigieren Sie den Code so, dass das Dokument wohlgeformt wird und die einzelnen Inhalte ihre Zugehörigkeit beibehalten.

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<FAHRRAD typ="Lastenrad">
```

```
  <modell marke = "Urban Arrow" >Cargo Line</modell>
```

```
  <motor>Bosch Performance Cargo Line
```

```
    <leistung>250 </ leistung>
```

```
    <antrieb>Riemen</antrieb>
```

```
</motor>
```

```
  <achszahl>2</achszahl>
```

```
</FAHRRAD>
```

Das ist erlaubt -> sind Zusatzinfos, IDs, Metadaten ...

```
<?xml version="1.0" encoding="UTF-8"?>

<BillingInformation>
  <Name>Right Plastic Products</Name>
  <BillingDate>2023-06-18</BillingDate>
  <Address>
    <Street>158 Edward st.</Street>
    <City>Brisbane</City>
    <State>QLD</State>
    <PostalCode>4000</PostalCode>
  </Address>
</BillingInformation>
```


JSON

- leichte Alternative zu XML
- Schlüssel-Wert-Paare
- zwei zentrale Strukturen: Objekte {} und Arrays []
- Werte:
- Strings, Zahlen, Boolean-Werte, Arrays oder Objekte

JSON example

```
{  
  "firstName": "John",  
  "lastName": "Smith",  
  "age": 25,  
  "address": {  
    "streetAddress": "21 2nd Street",  
    "city": "New York",  
    "state": "NY",  
    "postalCode": "10021"  
  },  
  "phone": [  
    {  
      "type": "home",  
      "number": "212 555-1234"  
    },  
    {  
      "type": "fax",  
      "number": "646 555-4567"  
    }  
  ]  
}
```

Sammlung von Schlüssel-Wert-Paaren }

Aufgabe 4 – Java-Objekte in JSON darstellen

Erstellen Sie eine einfache Student-Klasse. Dabei können Sie sich weiterhin an der ersten Aufgabe orientieren. Implementieren Sie eine toString-Methode, welche eine JSON-Darstellung der Objekte zurückgibt. Erstellen Sie ein paar Instanzen und geben Sie die String-Darstellung auf der Konsole aus. Erweitern Sie die Klasse um eine Liste von bestandenen Modulen. Worauf müssen Sie nun in der toString-Methode achten?

.formatted

Platzhalter	Datentyp	Beispiel
%s	String (Zeichenkette)	"%s".formatted("Hallo")
%d	Ganze Zahl (int, long, etc.)	"%d".formatted(42)
%f	<u>Fließkommazahl</u> (float, double)	"%f".formatted(3.14)
%b	Boolean (true / false)	"%b".formatted(true)
%n	<u>Plattformunabhängiger</u> <u>Zeilenumbruch</u>	<u>System.out.print("Zeile</u> <u>1%nZeile 2")</u>

```
{
    "id": 123456,
    "name": "Jane Doe",
    "modules": ["Math", "DB", "Java"]
}
```

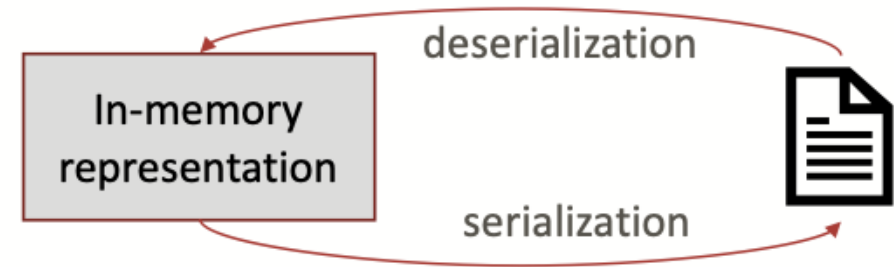
Serialisierung

Problem: Format is Java-spezifisch

- Wandeln Objekt in ein Format, das weitergegeben werden kann
- ObjectOutputStream

Deserialisierung

- Bytes werden wieder zu Java Objekt zusammengesetzt



1 Serialisierung

Aufgabe 1 – Serialisierung: Verständnis

Betrachten Sie das folgende Programm. Bei komplexen Anwendungen mit vielen, in un

```
import java.io.*;

void main() throws Exception {
    ObjectOutputStream os = new ObjectOutpu
    os.writeObject(new Student(123456, "John Doe", "john.doe@example.com"));
    os.close();
}
```

Problem: Format is Java-spezifisch

```
record Student(int id, String name, String mail) implements Serializable { }
```

Hiermit sagen wir:
Student darf serialisiert
werden

Führen Sie den Code aus. Wenn alles richtig funktioniert, sehen Sie in Ihrem Projektordner eine neue Datei „myobject.ser“. Verwenden Sie den folgenden Befehl in Ihrer Konsole, um den Inhalt der Datei zu betrachten.¹

```
hexdump -C myobject.ser
```

JSON/XML

```
String xml = ""  
<student>  
  <id>123456</id>  
  <name>Jane Doe</name>  
</student>  
"";
```

```
String json = ""  
{  
  "id": 123456,  
  "name": "Jane Doe"  
}  
"";
```

```
Files.writeString(Path.of("student.json"), json);
```

Java-Serialisierung:

Objekt -> Binärformat -> Datei

JSON/XML

Objekt -> Textdarstellung -> Datei

Aufgabe 2 – Serialisierung: DIY (Exkurs)

In dieser Aufgabe wollen wir die Serialisierung von Objekten selbstständig implementieren. Dafür können wir den record aus der vorherigen Aufgabe weiterverwenden. Schreiben Sie eine Methode `writeToDisk`, die eine Instanz der Klasse `Student` als Input erhält und dieses mithilfe von Reflection in einer Datei speichert. Überlegen Sie sich dafür ein eigenes (einfaches) Serialisierungsformat. Worauf müssen Sie achten, wenn Sie Datentypen mit variierender Länge (wie Strings) oder mit mehreren Elementen (wie Arrays) serialisieren wollen? Implementieren Sie außerdem eine Methode `readFromDisk`, welche einen Dateinamen als Argument erhält und Instanz der Klasse `Student` zurückgibt. Vergleichen Sie Ihre Implementierung und die Variante der Java-Standardbibliothek mit `hexdump`.

Wir wollen Selbst serialisieren:

Wie speichern wir Zahlen?

Wie speichern wir Strings?

Wo endet ein Feld?

Wie erkennen wir Arrays oder Listen?

```
ObjectOutputStream os = new ObjectOutputStream(...);
```

```
os.writeObject(student);
```

→ Java macht alles automatisch

Aufgabe 6 – JSON zu XML

Betrachten Sie den folgenden JSON-Codeausschnitt. Übersetzen Sie ihn so gut wie möglich nach XML.

```
{
  "name": "Jane Doe",
  "birthday": "2003-05-15",
  "major": "Information Systems",
  "degree": "Bachelor of Science",
  "passed": [
    {
      "name": "CS 101",
      "grade": "B+",
      "credits": 10
    },
    {
      "name": "Distributed Systems",
      "grade": "A-",
      "credits": 5
    },
    {
      "name": "Marketing",
      "grade": "C+",
      "credits": 5
    }
  ]
}
```

ca. 5 min

```
<student>
  <name>Jane Doe</name>
  <birthday>2003-05-15</birthday>
  <major>Information Systems</major>
  <degree>Bachelor of Science</degree>
  <passed>
    <module>
      <name>CS101</name>
      <grade>B+</grade>
      <credits>10</credits>
    </module>
    <module>
      <name>Distributed Systems</name>
      <grade>A-</grade>
      <credits>5</credits>
    </module>
    <module>
      <name>Marketing</name>
      <grade>C+</grade>
      <credits>5</credits>
    </module>
  </passed>
</student>
```

```
{
  "name": "Jane Doe",
  "birthday": "2003-05-15",
  "major": "Information Systems",
  "degree": "Bachelor of Science",
  "passed": [
    {
      "name": "CS 101",
      "grade": "B+",
      "credits": 10
    },
    {
      "name": "Distributed Systems",
      "grade": "A-",
      "credits": 5
    },
    {
      "name": "Marketing",
      "grade": "C+",
      "credits": 5
    }
  ]
}
```


GitHUb –
Code kommt
später

